

ER Triage System

Complete Project Task List & Implementation Guide

This document is your complete blueprint for building the Emergency Room Triage & Patient Management System from scratch in Java. Every feature is broken down by what it does, which data structures it touches, every method you need to implement, edge cases to handle, and the exact pitfalls to avoid. Follow the **Suggested Build Order** at the top — it reflects true dependency order so you are never blocked.

Suggested Build Order

Step	What to Build	Why First?	Priority
1	Patient class + MedicalHistoryNode	Every other class depends on Patient existing	CRITICAL
2	Custom Queue	Waiting room is needed for first demo	CRITICAL
3	Custom Stack	Undo must work before any destructive action	CRITICAL
4	Custom Hash Table	Lookup is used by nearly every other operation	CRITICAL
5	Custom Linked List (medical history)	Embedded in Patient; needed before admit works fully	HIGH
6	Custom BST	Sorted reports; can stub until core works	HIGH
7	EmergencyRoom orchestration class	Wires all structures together	HIGH
8	SortingAlgorithms class	One-time reports; independent utility class	MEDIUM
9	Main.java — CLI menu loop	UI wrapper around everything already built	MEDIUM
10	Testing & Benchmarking	Validate correctness + performance write-up	MEDIUM

PHASE 1 — CORE DATA STRUCTURES (Build these first, in order)

1. Patient Class + MedicalHistoryNode

CRITICAL

Overview

The Patient class is the fundamental data object the entire system is built around. It holds every piece of information about one patient and owns a reference to the head of their linked-list medical history. MedicalHistoryNode is the node class used inside that linked list — each node stores one medical event (visit, diagnosis, medication, allergy) and a pointer to the next node.

Java Files to Create

- Patient.java — the main patient data object
- MedicalHistoryNode.java — single node for the medical history linked list

Methods to Implement

Method	What It Does	Complexity
Patient(id, name, age, severity)	Constructor. Initialises all fields. Sets arrivalTime to current timestamp. Initialises historyHead to null.	O(1)
getId()	Returns the patient's unique ID string (format P001, P002...)	O(1)
getName() / getAge() / getSeverity()	Standard getters for display and sorting.	O(1)
getArrivalTime()	Returns LocalDateTime of when patient was admitted.	O(1)
addMedicalHistory(entry)	Creates a new MedicalHistoryNode and appends it to the END of the linked list.	O(h)
printMedicalHistory()	Traverses the linked list from head to tail, printing each node.	O(h)
toString()	Returns a formatted summary string for display in the CLI.	O(1)
MedicalHistoryNode(entry, next)	Constructor. Stores the entry string and pointer to next node.	O(1)

Implementation Hints

- Use `java.time.LocalDateTime.now()` for arrivalTime — store it at construction time.
- The linked list inside Patient does NOT need to be a separate LinkedList class yet — just a historyHead pointer directly in Patient works fine for Phase 1.
- Patient IDs should be auto-generated: keep a static counter in Patient or in EmergencyRoom and format with `String.format("P%03d", counter)`.
- Severity is an integer 1–4. Add a helper `getSeverityLabel()` that returns the string name (Emergency, Urgent, Delayed, Expectant) so the CLI can display it nicely.

Edge Cases to Handle

- Severity out of range (< 1 or > 4) — throw an `IllegalArgumentException` with a clear message.
- Null or blank name — reject at construction time.
- $\text{Age} \leq 0$ — reject at construction time.
- addMedicalHistory with null/empty string — skip silently or throw.
- printMedicalHistory when historyHead is null — print 'No medical history on record.'

■ COMMON PITFALLS — DON'T DO THIS

- Don't make Patient implement Comparable just yet — sorting logic belongs in SortingAlgorithms and BST, not in Patient.
- Don't use ArrayList for medical history inside Patient. The point is to build a real linked list.
- Don't store arrivalTime as a String — use LocalDateTime so you can compare arrival times in sort operations later.

2. Custom Queue — Waiting Room

CRITICAL

Overview

The Queue models the ER waiting room using strict FIFO (First In, First Out) ordering. Every time a patient is admitted they are enqueued. When a doctor calls the next patient they are dequeued. This is implemented from scratch — do NOT use Java's built-in LinkedList or ArrayDeque. Use a singly-linked node structure internally.

Java Files to Create

- Queue.java — custom generic FIFO queue backed by linked nodes
- QueueNode.java — (optional) inner class or separate file for queue nodes

Methods to Implement

Method	What It Does	Complexity
enqueue(Patient p)	Adds patient to the rear of the queue. Updates tail pointer.	O(1)
dequeue()	Removes and returns the patient at the front. Updates head pointer.	O(1)
peek()	Returns but does NOT remove the front patient. Returns null if empty.	O(1)
isEmpty()	Returns true if the queue has no elements.	O(1)
size()	Returns the current count of patients in the queue.	O(1)
contains(String id)	Returns true if a patient with the given ID is in the queue.	O(n)
display()	Prints all patients in queue order (front to back) without removing them.	O(n)

Implementation Hints

- Maintain two pointers: head (front, dequeue side) and tail (rear, enqueue side).
- Also keep an integer size field — increment on enqueue, decrement on dequeue. Saves an O(n) count.
- dequeue() must handle the case where head == tail (last element): set both to null after removal.
- The Queue should be typed for Patient specifically — generics are optional but cleaner.

Edge Cases to Handle

- dequeue() on empty queue — throw QueueUnderflowException or return null with a printed warning.
- peek() on empty queue — return null, do NOT throw.

- Enqueue the same patient twice — ideally check for duplicate ID first (use `contains()`).
- `size()` should always return the exact count, not a cached stale value.

■ COMMON PITFALLS — DON'T DO THIS

- Using an array-backed queue with fixed size — the waiting room size is unbounded. Use linked nodes.
- Forgetting to update tail when dequeuing the last element — tail will point to a removed node.
- Not maintaining the size counter — avoids an $O(n)$ traversal every time `size()` is called.

3. Custom Stack — Undo / Action Log

CRITICAL

Overview

The Stack is the undo mechanism for the entire system. Every action performed (admit, discharge, treat, call next patient) is recorded as an Action object and pushed onto the stack. When the user selects Undo, the top action is popped and reversed. This is LIFO (Last In, First Out). Build from scratch using linked nodes.

Java Files to Create

- `Stack.java` — custom LIFO stack
- `Action.java` — value object describing a reversible action

Methods to Implement

Method	What It Does	Complexity
<code>push(Action a)</code>	Pushes new action on top. Updates top pointer.	$O(1)$
<code>pop()</code>	Removes and returns top action. Returns null if empty.	$O(1)$
<code>peek()</code>	Returns top action without removing. Returns null if empty.	$O(1)$
<code>isEmpty()</code>	Returns true when stack has no elements.	$O(1)$
<code>size()</code>	Returns number of actions currently on stack.	$O(1)$
<code>Action(type, patientId, data)</code>	Constructor for the action record object. type = ADMIT / DISCHARGE / TREAT / CALL.	$O(1)$
<code>Action.getType() / getPatientId()</code>	Getters used by EmergencyRoom to determine how to reverse the action.	$O(1)$

Implementation Hints

- The Action class is a simple data container (value object) — it records enough info to reverse the operation. For ADMIT: store the full Patient object. For DISCHARGE: store the full Patient object. For TREAT: store patientId + the history entry that was added.
- Stack top is the most recently added node. top pointer always points to it.
- The EmergencyRoom class calls `stack.pop()` and then uses `action.getType()` in a switch statement to decide how to reverse the operation.
- You do NOT need to support multi-level redo — just undo is required.

Edge Cases to Handle

- `pop()` on empty stack — return null and print 'Nothing to undo.'

- Undo an ADMIT: must remove the patient from queue, hash table, BST, and array.
- Undo a DISCHARGE: must re-add the patient to hash table and BST (and queue if still waiting).
- Undo a TREAT: must remove the last medical history entry from the patient's linked list.
- Stack should not have a fixed capacity limit.

■ COMMON PITFALLS — DON'T DO THIS

- Using Java's built-in Stack class — build it from scratch.
- Storing only the patient ID in the Action — you need the full Patient object for undo-admit to work.
- Not clearing the action from the stack after the undo is performed.
- Assuming undo is always possible — check isEmpty() first and show a user-friendly message.

4. Custom Hash Table — O(1) Patient Lookup

CRITICAL

Overview

The hash table is the fastest lookup structure in the system. A doctor entering a patient ID must get the record in O(1) time — no scanning. The key is the patient ID string (e.g. P001), the value is the Patient object. You build the hash function yourself and handle collisions with chaining (linked lists at each bucket) or open addressing (linear probing). Chaining is recommended for this project.

Java Files to Create

- HashTable.java — the hash table with your own hash function
- HashEntry.java — key/value pair node used in chaining

Methods to Implement

Method	What It Does	Complexity
insert(String id, Patient p)	Hashes the key, finds the bucket, appends entry. Rehashes if load factor > 0.75.	O(1) avg
search(String id)	Hashes key, walks chain at bucket, returns Patient or null.	O(1) avg
delete(String id)	Hashes key, removes node from chain. Returns true if found.	O(1) avg
contains(String id)	Returns true if the ID exists. Used for duplicate checks.	O(1) avg
rehash()	Doubles table size, re-inserts all entries. Called automatically when load factor exceeded.	O(n)
size()	Returns the count of entries currently stored.	O(1)
getLoadFactor()	Returns size / capacity as a double. Used for benchmarking.	O(1)
hash(String key)	Private method. Your hash function. Returns an index in [0, capacity).	O(k)

Implementation Hints

→ A simple but effective hash for string keys: hash = 0; for each char c: hash = hash * 31 + c; return hash % capacity; — the prime 31 reduces collisions.

- Start with capacity = 16. Double when load factor > 0.75.
- For chaining: each bucket is the head of a linked list of `HashEntry` nodes. Each `HashEntry` holds (key, value, next).
- For collision resolution with chaining: on insert, if the bucket is non-empty, walk the chain to check for duplicate key first.
- Keep a separate `int count` field so `size()` is $O(1)$.

Edge Cases to Handle

- Insert with duplicate key — update the existing value, do NOT add a second entry.
- Search for a key that doesn't exist — return null cleanly.
- Delete the only entry in a chain — set bucket to null.
- Negative hash result from integer overflow — use `Math.abs()` or mask: `(hash & 0x7fffffff) % capacity`
- Rehash must correctly re-insert every existing entry into the new larger array.

■ COMMON PITFALLS — DON'T DO THIS

- Using Java's `HashMap` or `Hashtable` — build it from scratch.
- Not handling the case where `hash()` returns a negative number — causes `ArrayIndexOutOfBoundsException`.
- Forgetting to rehash — the table degrades to $O(n)$ as load factor climbs.
- Using `==` to compare String keys — always use `.equals()`.

5. Custom Linked List — Medical History

HIGH

Overview

Each `Patient` object owns a linked list of medical history entries. This list is append-only in normal use (new entries added to the end) but must support deletion of a specific index (for corrections). The list is singly-linked. You may implement it as inner logic inside `Patient` itself using `MedicalHistoryNode`, or as a standalone `LinkedList` class that `Patient` instantiates.

Java Files to Create

- `LinkedList.java` — optional standalone class; may also live inside `Patient.java`
- `MedicalHistoryNode.java` — already created in Feature 1

Methods to Implement

Method	What It Does	Complexity
<code>addHistory(String entry)</code>	Creates a new node and appends it to the tail of the list.	$O(h)$
<code>getHistory()</code>	Returns a formatted String of all entries, numbered and in order.	$O(h)$
<code>deleteHistory(int index)</code>	Removes the node at position <code>index</code> (0-based). Reconnects pointers.	$O(h)$
<code>getHistoryHead()</code>	Returns the head node reference — used by BST/reports if needed.	$O(1)$
<code>historySize()</code>	Returns number of entries in the history list.	$O(h)$

Implementation Hints

- Traverse from head each time — no tail pointer is strictly necessary unless you want $O(1)$ append.
- For `deleteHistory(0)` (deleting head): update `historyHead = historyHead.next`.
- For `deleteHistory` at index > 0 : traverse to node at index-1, then `prev.next = prev.next.next`.
- Number entries when printing: 'Visit 1: ... Visit 2: ...' for readability.

Edge Cases to Handle

- `deleteHistory` on an out-of-range index — print an error, do nothing.
- `deleteHistory` on an empty list — handle gracefully.
- `historySize()` on empty list — return 0, don't throw a `NullPointerException`.
- `getHistory()` on empty list — return a user-friendly message, not an empty string.

■ COMMON PITFALLS — DON'T DO THIS

- Using `ArrayList` inside `Patient` to fake a linked list — the grader will check.
- Not connecting `prev.next` correctly on delete — you will corrupt the chain.
- Off-by-one in `deleteHistory` — sketch the pointer operations on paper before coding.

6. Binary Search Tree — Sorted Reports

HIGH

Overview

The BST mirrors the patient database keyed by last name. Its only job is to produce an alphabetically sorted list of all active patients in $O(n)$ time via in-order traversal — no re-sorting needed. It is updated (insert/delete) whenever patients are admitted or discharged. The BST is not required to self-balance for this project, but you must handle the standard 3-case deletion correctly.

Java Files to Create

- `BinarySearchTree.java` — the BST with insert, delete, traversal
- `BSTNode.java` — tree node holding the key and `Patient` reference

Methods to Implement

Method	What It Does	Complexity
<code>insert(String lastName, Patient p)</code>	Inserts a new node in correct BST position.	$O(\log n)$ avg
<code>delete(String lastName)</code>	Removes a node. Handles 3 cases: leaf, one child, two children.	$O(\log n)$ avg
<code>search(String lastName)</code>	Returns the <code>Patient</code> if found, null otherwise.	$O(\log n)$ avg
<code>inOrder()</code>	Traverses left-root-right. Returns a List in alphabetical order.	$O(n)$
<code>inOrderPrint()</code>	Same traversal but prints directly to stdout.	$O(n)$
<code>isEmpty()</code>	Returns true if root is null.	$O(1)$
<code>height()</code>	Returns the height of the tree — used in benchmarking section.	$O(n)$

Implementation Hints

- BST key = patient last name (lowercase for case-insensitive compare). Extract last name with:
`name.substring(name.lastIndexOf(" ") + 1).toLowerCase()`.
- For two-children deletion: find the in-order successor (leftmost node of right subtree), copy its key/value into the current node, then delete the successor.
- In-order traversal using recursion: `inOrder(node.left) → visit(node) → inOrder(node.right)`.
- If two patients share a last name, append a tiebreaker (patient ID) to the key: "smith_P001".

Edge Cases to Handle

- Delete a node with two children — use the in-order successor strategy.
- Delete from an empty tree — return false, do not throw.
- Insert a duplicate key (same last name + same ID) — check before inserting.
- Search in an empty tree — return null immediately.
- inOrder() on empty tree — return an empty list, not null.

■ COMMON PITFALLS — DON'T DO THIS

- Using Java's TreeMap — build the BST from scratch.
- Only handling the leaf case in delete() and ignoring the two-children case — you will lose points.
- Losing the BST property after deletion — always verify with an in-order print that the list is still sorted.
- Case-sensitive string comparison — 'Smith' and 'smith' should be treated as the same key.

PHASE 2 — SYSTEM INTEGRATION (Wire the structures together)

7. EmergencyRoom Class — System Orchestrator

HIGH

Overview

EmergencyRoom is the central manager that owns all data structures and wires them together. Every menu operation in Main.java calls a method on EmergencyRoom. This class does NOT implement any data structure itself — it delegates. It holds: the patient array (database), a Queue (waiting room), a Stack (action log), a HashTable (lookup index), and a BST (sorted reports).

Java Files to Create

- EmergencyRoom.java — the system orchestrator class

Methods to Implement

Method	What It Does	Complexity
EmergencyRoom(int capacity)	Constructor. Initialises all structures. Sets bedCapacity.	O(1)
admitPatient(name, age, severity, symptoms)	Creates Patient, adds to array + queue + hashTable + BST. Pushes ADMIT to stack.	O(log n)
callNextPatient()	Dequeues from waitingRoom. Retrieves full record from hashTable. Displays info.	O(1)
lookupPatient(String id)	Calls hashTable.search(id). Returns and displays Patient.	O(1)

<code>viewMedicalHistory(String id)</code>	Looks up patient, calls <code>patient.printMedicalHistory()</code> .	O(h)
<code>addTreatmentNote(id, note)</code>	Looks up patient, calls <code>addMedicalHistory(note)</code> . Pushes TREAT to stack.	O(1)
<code>undoLastAction()</code>	Pops stack. Inspects Action type. Reverses operation on affected structures.	O(log n)
<code>generateReport(int sortType)</code>	1 = BST in-order. 2 = sort by arrival time. 3 = sort by severity.	O(n log n)
<code>dischargePatient(String id)</code>	Removes from hashTable + BST + array. Pushes DISCHARGE to stack.	O(log n)
<code>getStatistics()</code>	Prints total patients, queue length, severity breakdown.	O(n)
<code>isFull()</code>	Returns true if <code>patientDatabase</code> array is at <code>bedCapacity</code> .	O(1)

Implementation Hints

→ The patient array serves as the permanent bed-slot record. Use a simple integer counter to track the next open index. When a patient is discharged, set their slot to null (don't shift the array).

→ `admitPatient` must call all of: `array[nextSlot] = p`, `waitingRoom.enqueue(p)`, `hashTable.insert(p.getId(), p)`, `bst.insert(lastName, p)`, `actionStack.push(new Action(ADMIT, p))`.

→ For `generateReport` type 2 and 3: extract all non-null patients from the array into a temporary list, then pass to `SortingAlgorithms`. Do not modify the array itself.

→ `undoLastAction`: use a switch on `action.getType()` — each case knows exactly how to reverse itself.

Edge Cases to Handle

- `admitPatient` when `isFull()` — print 'ER at full capacity. Cannot admit more patients.' and return.
- `callNextPatient` when `queue.isEmpty()` — print 'No patients waiting.' and return.
- `lookupPatient` with ID that doesn't exist — print 'Patient not found.'
- `dischargePatient` on a patient still in the waiting queue — handle gracefully (remove from queue too).
- `undoLastAction` on an empty stack — print 'No actions to undo.'

■ COMMON PITFALLS — DON'T DO THIS

- Putting data structure logic (hashing, tree balancing) inside `EmergencyRoom` — keep it thin; it only delegates.
- Forgetting to update the BST on discharge — the sorted report will show discharged patients.
- Not pushing every operation to the action stack — undo will silently fail for skipped operations.

8. SortingAlgorithms Class — One-Time Reports

MEDIUM

Overview

A standalone utility class with static sort methods. Used when generating reports by a field that doesn't have a dedicated BST (e.g. arrival time, severity score). You must implement at least two sorting algorithms from scratch. The recommended pair is Selection Sort (simple, good for small n) and Quicksort (fast, $O(n \log n)$ average). Bubble Sort and Insertion Sort are also acceptable for the second algorithm.

Java Files to Create

- `SortingAlgorithms.java` — static utility class with multiple sort implementations

Methods to Implement

Method	What It Does	Complexity
<code>selectionSort(Patient[] arr, Comparator cmp)</code>	Selection sort implementation. In-place, $O(n^2)$.	$O(n^2)$
<code>bubbleSort(Patient[] arr, Comparator cmp)</code>	Bubble sort. In-place, $O(n^2)$. Add early-exit optimisation.	$O(n^2)$
<code>insertionSort(Patient[] arr, Comparator cmp)</code>	Insertion sort. $O(n^2)$ worst, $O(n)$ on nearly-sorted.	$O(n^2)$
<code>quickSort(Patient[] arr, int lo, int hi, Comparator cmp)</code>	Quicksort. Recursive, $O(n \log n)$ average.	$O(n \log n)$
<code>sortByArrivalTime(Patient[] arr)</code>	Convenience method: calls <code>quickSort</code> with a time comparator.	$O(n \log n)$
<code>sortBySeverity(Patient[] arr)</code>	Convenience method: calls <code>quickSort</code> with severity comparator.	$O(n \log n)$

Implementation Hints

- Use a `Comparator` parameter so each method can sort by any field — avoids writing a separate sort for every field.
- Quicksort pivot strategy: median-of-three is better than always picking the last element — prevents $O(n^2)$ on already-sorted input.
- To compare arrival times: `p1.getArrivalTime().compareTo(p2.getArrivalTime())`.
- To compare severity: lower number = higher urgency, so sort ascending by integer value.
- For benchmarking: record `System.nanoTime()` before and after each sort on the same array copy.

Edge Cases to Handle

- Null entries in the array (discharged bed slots) — filter them out before passing to sort.
- Array of length 0 or 1 — return immediately without sorting.
- All patients with the same severity — sort is stable (preserves original order); test this.
- Quicksort with empty sub-array (`lo > hi`) — base case must handle this.

■ COMMON PITFALLS — DON'T DO THIS

- Using Java's `Arrays.sort()` or `Collections.sort()` — implement from scratch.
- Not copying the array before benchmarking — each algorithm must run on identical input.
- Choosing first or last element as pivot always — leads to $O(n^2)$ on sorted data. Use median-of-three.
- Sorting the live patient database array in place — always sort a copy for reporting.

9. Main.java — Command-Line Interface & Menu Loop

MEDIUM

Overview

Main.java is the program entry point. It creates one EmergencyRoom instance and drives the interaction loop. It reads user input, maps choices to EmergencyRoom method calls, and handles all input validation. Keep Main thin — it should contain almost no business logic. Its only job is to parse input and call the right method.

Java Files to Create

- Main.java — entry point with main(), menu loop, and input parsing

Methods to Implement

Method	What It Does	Complexity
main(String[] args)	Creates EmergencyRoom. Calls showMenu() in a loop until exit.	—
showMenu()	Prints the numbered menu options.	O(1)
getIntInput(Scanner sc, String prompt, int min, int max)	Helper: reads and validates an integer in range. Reprompts on invalid input.	O(1)
getStringInput(Scanner sc, String prompt)	Helper: reads a non-empty string. Reprompts on blank.	O(1)
handleAdmit(Scanner sc, EmergencyRoom er)	Collects name, age, severity, symptoms. Calls er.admitPatient().	O(1)
handleReport(Scanner sc, EmergencyRoom er)	Shows report sub-menu. Calls er.generateReport(choice).	O(1)

Implementation Hints

- Wrap the entire menu loop in a while (true) — break only when the user picks 9 (Exit).
- Use a single Scanner(System.in) instance — do NOT create a new Scanner on every prompt.
- Always call sc.nextLine() after sc.nextInt() to consume the newline character before reading a String.
- Print a clear separator (e.g. a row of dashes) between menu operations so output is readable during the demo.

Edge Cases to Handle

- User enters a letter when a number is expected — catch InputMismatchException, consume the bad token, and reprompt.
- User enters a severity of 0 or 5 — rejects with 'Must be 1–4.'
- User enters an empty name — rejects and reprompts.
- Exit while queue is not empty — optionally warn the user: 'There are still X patients waiting.'

■ COMMON PITFALLS — DON'T DO THIS

- Putting ER logic in Main — Main only reads input and calls methods.
- Creating a new Scanner inside each helper method — causes stdin to be consumed unpredictably.
- Not handling InputMismatchException — one bad keystroke crashes the program during the demo.

PHASE 3 — TESTING & DOCUMENTATION (Validate and write up)

10. Testing & Benchmarking

MEDIUM

Overview

Testing is not optional — it is what turns working code into a trustworthy system. You do not need a formal testing framework (JUnit is fine if you know it, but not required). Write a dedicated test sequence that exercises every feature and every edge case. The benchmarking section compares your sorting algorithms empirically on the same dataset at three scales: 100, 1000, and 10000 patients.

Java Files to Create

- `TestRunner.java` — optional but recommended: a main class that runs scripted test scenarios
- `BENCHMARKS.md` — written analysis of sort performance results

Methods to Implement

Method	What It Does	Complexity
<code>testAdmitAndQueue()</code>	Admit 5 patients. Verify queue size = 5. Call next. Verify queue size = 4.	—
<code>testHashTableLookup()</code>	Admit 10 patients. Look up each by ID. Verify all returned correctly.	—
<code>testUndoAdmit()</code>	Admit P001, P002. Undo. Verify P002 is gone from all structures.	—
<code>testUndoDischarge()</code>	Admit P001. Discharge. Undo. Verify P001 is back in hashTable and BST.	—
<code>testBSTInOrder()</code>	Admit Alice, Charlie, Bob. Generate sorted report. Verify: Alice, Bob, Charlie.	—
<code>testSortBySeverity()</code>	Admit patients with severity 3,1,4,2. Sort. Verify order is 1,2,3,4.	—
<code>testCapacityLimit()</code>	Set capacity to 3. Admit 4 patients. Verify 4th is rejected.	—
<code>benchmarkSorts(int n)</code>	Generate n random patients. Run each sort. Record and print elapsed nanoseconds.	—

Implementation Hints

- For benchmarks: generate synthetic patient data (random names, severities) in a loop. Use `System.nanoTime()` not `System.currentTimeMillis()` — nanosecond precision matters for small n.
- Sort a COPY of the array for each algorithm: `Patient[] copy = Arrays.copyOf(arr, arr.length)` — then all algorithms run on identical input.
- For `BENCHMARKS.md`: record results in a table (n | SelectionSort (ms) | BubbleSort (ms) | QuickSort (ms)) and explain the observed trends.
- To stress-test the hash table: insert 1000 patients and verify load factor is being managed (rehash triggered).

Edge Cases to Handle

- Test undo with no previous actions — verify 'Nothing to undo' message appears.
- Test `lookupPatient` with a non-existent ID — verify null handling.
- Test medical history with 0 entries — verify 'No history on record' is shown.

- Test queue isEmpty() dequeue — verify exception/null handling.

■ **COMMON PITFALLS — DON'T DO THIS**

- Only testing the happy path — graders specifically look for edge case handling.
- Running benchmarks only once — results vary due to JVM warmup; run each sort 3 times and average.
- Using the same array for all sort benchmarks without resetting — each algorithm must sort an identical unsorted copy.

Final Submission Checklist

STRUCTURE

- Patient.java + MedicalHistoryNode.java exist and compile
- Queue.java — custom, backed by linked nodes (no built-ins)
- Stack.java + Action.java — custom, with all undo logic
- HashTable.java — custom hash function + chaining collision resolution
- LinkedList for medical history — real linked nodes, no ArrayList
- BinarySearchTree.java — insert, delete (3 cases), in-order traversal
- SortingAlgorithms.java — at least 2 algorithms from scratch
- EmergencyRoom.java — orchestrates all structures
- Main.java — working CLI with all 9 menu options

FUNCTIONALITY

- Admit patient → updates queue, hash table, BST, array
- Call next patient → dequeues correctly
- Look up by ID → $O(1)$ hash table hit
- View medical history → traverses linked list
- Undo last action → reverses every affected structure
- Generate sorted report (all 3 sort options work)
- Discharge patient → removes from hash table + BST + array
- View statistics → shows queue size, total patients, severity breakdown

QUALITY

- All edge cases handled without crashes
- No use of Java built-in data structures where custom is required
- Benchmarking results documented in BENCHMARKS.md
- Code is readable with meaningful variable names and comments
- Each class has a brief Javadoc comment explaining its role

Build in order. Test as you go. Never open the IDE until you can explain the logic on paper.